

Introduction to Distributed Computing Algorithms

Distributed Computing Algorithms are methods used to manage communication, coordination, synchronization, decision-making, and recovery among multiple computers connected through a network. In distributed systems, multiple nodes work together to achieve a common objective while appearing as a single system.

These algorithms ensure:

- Reliable communication
- Coordination among nodes
- Data consistency
- Fault handling
- Recovery mechanisms

1. Communication and Coordination in Distributed Systems

Communication and Coordination refer to mechanisms through which distributed nodes exchange information and synchronize activities to perform tasks collectively. Since distributed systems consist of multiple independent computers, communication becomes necessary for:

- Sharing data
- Synchronizing tasks
- Resource management
- Decision making

Coordination ensures that all nodes work correctly without conflicts.

Communication Models

A. Message Passing

Nodes exchange information using messages.

Example: Node A → Message → Node B

Used in:

- Distributed databases
- Cloud computing

B. Remote Procedure Call (RPC)

Allows one computer to execute procedures located on another system.

Example: Client → Request → Server

C. Publish–Subscribe Model

Publisher sends data.

Subscribers receive updates.

Example: Publisher → Subscribers

Used in:

- Notification systems
- Streaming platforms

Coordination Mechanisms

1. Synchronization - Maintains execution order.

2. Mutual Exclusion - Allows only one process to access shared resources.

3. Leader Election - Selects one coordinator node.

4. Clock Synchronization - Maintains consistent timing.

Advantages

1. Efficient resource utilization.
2. Supports scalability.
3. Improves collaboration.

Disadvantages

1. Communication delay.
2. Complex coordination.

Applications

- Distributed databases
- Multi-server applications

2. Distributed Consensus Algorithms

Distributed Consensus Algorithms are algorithms that help multiple nodes agree on a single value or system state despite failures.

Consensus ensures:

- Data consistency
- Reliability
- Fault tolerance

Without consensus:

- Data conflicts occur
- System becomes inconsistent

General Working

Step 1: Node proposes value

Step 2: Nodes exchange messages

Step 3: Agreement achieved

Step 4: Value committed

A. Viewstamped Replication (VR)

Viewstamped Replication is a consensus protocol that maintains system consistency using replicas.

Working

1. Primary node receives request.
2. Replicas receive updates.
3. Commit operation.
4. Elect new primary if failure occurs.

Advantages

- High availability
- Strong consistency

Applications

- Distributed databases
- Replicated systems

B. RAFT Algorithm

RAFT is a distributed consensus algorithm designed for simplicity and fault tolerance

Components of RAFT

- 1. Leader:** Handles requests.
- 2. Followers:** Receive updates.
- 3. Candidate:** Attempts leadership.

Working of RAFT

Step 1: Leader elected.

Step 2: Leader receives requests.

Step 3: Log replication.

Step 4: Commit operation.

Advantages

1. Easy implementation.
2. Strong consistency.
3. Fault tolerant.

Disadvantages

1. Leader dependency.

Applications

- Distributed storage
- Cloud platforms

C. ZAB (ZooKeeper Atomic Broadcast)

ZAB is a consensus protocol used to maintain consistency in replicated distributed systems.

Working

1. Leader elected.
2. Transactions ordered.
3. Updates broadcast.
4. State synchronized.

Features

- Atomic delivery
- Ordering guarantee
- Crash recovery

Applications

- Coordination services
- Distributed management

D. Mencius Algorithm

Mencius is a consensus algorithm designed to improve throughput by allowing multiple leaders.

Working

- Multiple nodes become leaders.
- Requests processed simultaneously.
- Consensus maintained.

Advantages

1. Higher parallelism.
2. Better performance.

Disadvantages

1. Complex coordination.

Applications

- Large-scale systems

E. Paxos and Variants

Paxos is a distributed consensus algorithm used to achieve agreement among unreliable nodes.

Working of Paxos

Roles

- Proposer
- Acceptor
- Learner

Process

Propose -> Accept -> Commit

Variants of Paxos

1. Fast Paxos - Reduces communication delay.

Features

- Faster consensus
- Lower latency

2. Egalitarian Paxos (EPaxos) - Allows multiple leaders.

Features

- Parallel execution
- Improved performance

Advantages of Paxos

1. High reliability.
2. Fault tolerance.

Disadvantages

1. Complex implementation.

Comparison of Consensus Algorithms

Algorithm	Leader	Fault Tolerance	Performance
Viewstamped Replication	Single	High	Medium
RAFT	Single	High	High
ZAB	Single	High	High
Mencius	Multiple	High	Very High
Paxos	Multiple	Very High	Complex

3. Fault Tolerance and Recovery in Distributed Systems

Fault Tolerance is the ability of a distributed system to continue functioning even when failures occur. Recovery restores normal operation after failure.

Failures may occur due to:

- Network issues
- Hardware failure
- Software crash
- Data corruption

Fault tolerance ensures service continuity.

Types of Failures

1. Crash Failure - Node stops functioning.

2. Omission Failure - Messages are lost.

3. Timing Failure - Operation exceeds expected time.

4. Byzantine Failure - Node behaves unpredictably.

Fault Tolerance Techniques

A. Replication - Maintain multiple copies of data.

B. Checkpointing - Save system state periodically.

C. Redundancy - Use backup components.

D. Load Balancing - Distribute workload.

Recovery Mechanisms

1. Rollback Recovery - Restore previous checkpoint.

2. Forward Recovery - Move directly to safe state.

3. Restart Recovery - Restart failed node.

Working of Recovery

Failure Detected -> Isolation -> Recovery Process -> Restore State -> Resume Operation

Advantages

1. Increases availability.
2. Prevents service interruption.
3. Improves reliability.

Disadvantages

1. Additional storage cost.
2. Recovery overhead.

Applications

- Cloud computing

- Banking systems

Introduction to Load Balancing and Resource Allocation

Load Balancing is the process of distributing workload among multiple servers, processors, or computing resources to achieve efficient utilization and avoid overload.

Resource Allocation is the process of assigning available computational resources such as CPU, memory, storage, and bandwidth to different tasks efficiently.

Need for Load Balancing

In distributed systems:

- Workload may become uneven.
- Some servers may overload.
- Some resources remain idle.

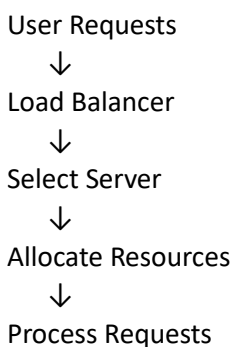
Load balancing ensures:

- Better performance
- High availability
- Efficient resource usage
- Reduced response time

Objectives of Load Balancing

1. Maximize throughput
2. Minimize response time
3. Avoid resource overload
4. Improve scalability
5. Maintain fault tolerance

Load Balancing Process



1. Weighted Round Robin (WRR)

Weighted Round Robin is a load balancing strategy in which requests are distributed among servers according to assigned weights.

Servers with higher weights receive more requests.

Unlike simple Round Robin, WRR considers server capacity.

Higher capacity server

→ Higher weight

→ More requests

Lower capacity server

→ Lower weight

→ Fewer requests

Working

Suppose:

Server Weight

Server A 5

Server B 3

Server C 2

Requests:

A → A → A → A → A

B → B → B

C → C

Cycle repeats.

Algorithm Steps

Step 1: Assign weights.

Step 2: Receive request.

Step 3: Allocate according to weight.

Step 4: Repeat.

Advantages

1. Better resource utilization.
2. Supports heterogeneous servers.
3. Easy implementation.

Disadvantages

1. Static weight assignment.
2. Cannot detect real-time load.

Applications

- Cloud computing

- Web servers

2. Least Connection Load Balancing

Least Connection assigns new requests to the server with the minimum active connections.

This strategy assumes:

Lower connections → Lower load

Higher connections → Higher load

Working

Example:

Server Active Connections

A 25

B 10

C 18

New request → Server B

Algorithm

Step 1: Check active connections.

Step 2: Select minimum.

Step 3: Assign request.

Advantages

1. Dynamic distribution.
2. Efficient for variable workloads.

Disadvantages

1. Connection count may not represent actual load.

Applications

- Web hosting
- Database servers

3. Randomized Load Balancing

Randomized Load Balancing distributes incoming requests randomly among available servers. No load information is used. Each server receives requests based on random selection.

Working

Example:

Request 1 → Server C

Request 2 → Server A

Request 3 → Server B

Algorithm

Generate random number → Select server → Assign request

Advantages

1. Very simple implementation.
2. Low overhead.

Disadvantages

1. Uneven distribution possible.
2. Performance may reduce.

Applications

- Small distributed systems
- Experimental environments

4. Dynamic Load Balancing

Dynamic Load Balancing allocates workload according to real-time resource availability. Unlike static methods, decisions change continuously.

Parameters considered:

- CPU usage
- Memory usage
- Network traffic
- Queue size

Working

Step 1: Monitor resources.

Step 2: Calculate current load.

Step 3: Assign request dynamically.

Step 4: Repeat continuously.

Example

If Server A becomes overloaded:

Requests → Redirect to Server B

Advantages

1. High performance.
2. Adaptive allocation.
3. Better scalability.

Disadvantages

1. Complex implementation.
2. Monitoring overhead.

Applications

- Cloud platforms
- AI systems
- Big data systems

5. Centralized Load Balancing

Centralized Load Balancing uses a single controller to manage workload distribution.

A central node:

- Monitors resources
- Makes decisions
- Allocates requests

Architecture

Clients -> Central Load Balancer -> Servers

Working

Controller receives request -> Analyze server status -> Assign workload.

Advantages

1. Easy control.
2. Simple management.

Disadvantages

1. Single point of failure.
2. Limited scalability.

Applications

- Enterprise systems
- Private cloud

6. Distributed Load Balancing

Distributed Load Balancing distributes decision-making across multiple nodes. No central controller exists. Each node participates in load balancing.

Architecture

Node A ↔ Node B ↔ Node C

All nodes coordinate.

Working

Each node:

- Monitors own load
- Exchanges information
- Shares workload

Advantages

1. Fault tolerant.
2. Highly scalable.
3. Eliminates central failure.

Disadvantages

1. Complex coordination.
2. Communication overhead.

Applications

- Cloud computing
- Edge computing
- Distributed databases

7. Predictive Load Balancing

Predictive Load Balancing uses historical data and intelligent models to predict future workloads and allocate resources in advance.

This approach uses:

- Machine Learning
- Artificial Intelligence
- Historical usage data

to prevent overload before it occurs.

Working

Step 1: Collect historical data.

Step 2: Train prediction model.

Step 3: Forecast future load.

Step 4: Allocate resources.

Example

Traffic prediction:

Expected spike at 8 PM

↓

Additional servers activated.

Advantages

1. Prevents overload.
2. Better performance.
3. Efficient resource utilization.

Disadvantages

1. Requires prediction models.
2. Higher implementation complexity.

Applications

- AI cloud systems
- Streaming platforms
- Smart infrastructure

Comparison of Load Balancing Strategies

Strategy	Decision Basis	Dynamic	Complexity
Weighted Round Robin	Assigned weights	No	Low
Least Connection	Active connections	Yes	Medium
Randomized	Random selection	No	Low
Dynamic	Current load	Yes	High
Centralized	Central controller	Partial	Medium
Distributed	Multiple nodes	Yes	High
Predictive	Future prediction	Yes	Very High

Benefits of Load Balancing

1. Improves system performance.
2. Reduces response time.
3. Supports scalability.
4. Prevents overload.
5. Enhances fault tolerance.

1. Machine Learning for Resource Allocation

Machine Learning (ML) for Resource Allocation uses data-driven models to allocate computing resources

efficiently based on system behavior and workload patterns.

Resource allocation means assigning:

- CPU
- Memory
- Storage
- Network bandwidth

Traditional allocation methods use predefined rules.

Machine Learning learns resource demand automatically.

Working

Step 1: Collect historical workload data.

Step 2: Train ML model.

Step 3: Predict future resource requirements.

Step 4: Allocate resources.

Machine Learning Techniques Used

A. Supervised Learning

Predict future resource demand.

Examples:

- Regression
- Decision Trees

B. Unsupervised Learning

Detect usage patterns.

Examples:

- Clustering

C. Deep Learning

Handle large-scale resource optimization.

Example

Cloud Platform:

High traffic predicted

↓

Allocate additional CPU and memory

↓

Prevent overload

Advantages

1. Efficient resource utilization.
2. Reduces operational cost.
3. Improves scalability.
4. Supports automatic decisions.

Disadvantages

1. Requires historical data.
2. Model training overhead.

Applications

- Cloud computing
- Data centers
- Virtual machines
- Big data platforms

2. Reinforcement Learning for Dynamic Load Balancing

Reinforcement Learning (RL) is an AI technique where an agent learns optimal actions through rewards and penalties.

In distributed computing, RL is used to dynamically balance workload.

Traditional load balancing uses fixed rules.

Reinforcement Learning:

- Learns continuously
- Adapts to changing environments
- Optimizes decisions

Components of RL

Agent

Load balancing controller.

Environment

Distributed system.

State

Current workload condition.

Action

Assign task.

Reward

Performance improvement.

Working

Step 1:

Observe current system state.

Step 2:

Select action.

Step 3:

Receive reward.

Step 4:

Update policy.

Repeat.

Example

Server A overloaded

RL redirects requests

Response time decreases

Positive reward

Advantages

1. Adaptive load balancing.
2. Reduces latency.
3. Improves throughput.

Disadvantages

1. Requires training time.
2. Complex implementation.

Applications

- Cloud systems
- Smart networks
- Edge computing

3. Genetic Algorithms for Task Scheduling

Genetic Algorithm (GA) is an evolutionary optimization technique used to schedule tasks efficiently across distributed resources.

Task scheduling determines:

- Which task runs
- Where it executes
- When it executes

Genetic Algorithm searches for optimal schedules.

Working

Step 1: Generate initial schedules.

Step 2: Evaluate fitness.

Step 3: Select best schedules.

Step 4: Apply crossover.

Step 5: Apply mutation.

Step 6: Generate improved schedule.

Example

Tasks:

T1

T2

T3

Servers:

S1

S2

S3

GA generates optimal mapping.

Fitness Parameters

- Execution time
- Energy usage
- Cost
- Resource utilization

Advantages

1. Finds near-optimal schedules.
2. Handles large search space.
3. Flexible optimization.

Disadvantages

1. Slow convergence.
2. Computational cost.

Applications

- Grid computing
- Cloud scheduling

4. Swarm Intelligence for Distributed Optimization

Swarm Intelligence is an AI technique inspired by collective behavior of biological systems such as ants, birds, and bees. It is used for distributed optimization.

Swarm Intelligence solves problems through cooperation among simple agents.

Popular approaches:

- Ant Colony Optimization (ACO)
- Particle Swarm Optimization (PSO)

Working

Step 1: Initialize agents.

Step 2: Explore search space.

Step 3: Share information.

Step 4: Update solutions.

Step 5: Find optimal solution.

A. Ant Colony Optimization (ACO)

Inspired by ants finding shortest paths.

Used for:

- Routing
- Scheduling

B. Particle Swarm Optimization (PSO)

Inspired by bird movement.

Used for:

- Resource optimization
- Task assignment

Example

Multiple servers cooperate.

↓

Workload distributed.

↓

Performance improves.

Advantages

1. Distributed decision-making.
2. Robust optimization.
3. High scalability.

Disadvantages

1. Parameter tuning required.
2. May converge slowly.

Applications

- Network optimization

- Cloud computing

Comparison of AI Techniques in Distributed Computing

Technique	Main Purpose	Learning Type	Major Application
Machine Learning	Resource Allocation	Data-driven	Cloud systems
Reinforcement Learning	Dynamic Load Balancing	Reward-based	Adaptive balancing
Genetic Algorithm	Task Scheduling	Evolutionary	Scheduling
Swarm Intelligence	Distributed Optimization	Collective behavior	Routing

Benefits of AI in Distributed Computing

1. Intelligent decision making.
2. Better resource utilization.
3. Reduced execution time.
4. Improved scalability.
5. Higher system performance.